TP — Site vitrine d'un commerce (HTML / CSS)

Présentation

Vous allez construire, **seul**, le site vitrine d'un commerce de votre choix : 3 pages en HTML et CSS, sans JavaScript ni PHP.

Ce site vous servira de nouveau plus tard : au module PHP, vous le rendrez dynamique (menu commun à toutes les pages, catalogue généré à partir de données, formulaire de contact qui fonctionne). Plus votre HTML sera propre et régulier maintenant, plus cette étape sera simple.

Cahier des charges

Choisissez un commerce local fictif ou réel : boulangerie, fleuriste, caviste, librairie, salon de coiffure, magasin de vélos, food-truck… Donnez-lui un nom, un slogan et une identité (2 à 3 couleurs, 1 ou 2 polices). Validez votre choix avec l'enseignant avant de coder.

Page	Fichier	Contenu obligatoire
Accueil	`index.html`	Bannière avec nom et slogan, texte de présentation, 3 points forts du commerce, aperçu de 3 produits ou services
Produits / services	`produits.html`	Au moins 6 produits ou services, chacun présenté sous forme de « carte » : image, nom, courte description, prix
Contact	`contact.html`	Adresse, horaires d'ouverture, téléphone, lien vers le plan, formulaire de contact

Contraintes techniques :

- un seul fichier CSS principal (`style.css`) et une feuille dédiée au responsive (`responsive.css`) ; aucun attribut `style` dans le HTML ;
- les 3 pages partagent exactement le même en-tête, le même menu et le même pied de page ;
- noms de fichiers et d'images en minuscules, sans accents ni espaces (`-` ou `_` autorisés) ;
- images légères (moins de 300 Ko chacune), toutes avec un attribut `alt` ;
- code indenté et commenté ;
- le site doit s'afficher correctement sur ordinateur et sur smartphone, et être testé sur au moins 2 navigateurs.

Étape 0 — Mettre en place le projet

Créez un dossier au nom de votre commerce et reproduisez exactement cette arborescence :

```
nom-du-commerce/
├── index.html
├── produits.html
├── contact.html
├── css/
│   ├── style.css
│   └── responsive.css
└── images/
    ├── logo.png
    └── ...
```

1. Ouvrez le dossier dans votre éditeur de code (PhpStorm, VS Code, Sublime Text…).
2. Récupérez 8 à 10 images libres de droits (Unsplash, Pexels) ; renommez-les proprement (`pain-campagne.jpg`, pas `IMG_4521 (2).JPG`) et réduisez leur poids si besoin.
3. Notez dans un commentaire en haut de `style.css` vos couleurs (en hexadécimal) et vos polices.

Étape 1 — Structure HTML commune

Toutes vos pages reposent sur le même squelette. Construisez-le une fois dans `index.html`, vérifiez-le, puis copiez-le dans les deux autres pages.

1. Écrivez la structure de base : doctype, `html`, `head` (charset UTF-8, titre), `body`.
2. Dans le `head`, reliez vos deux feuilles de style avec la balise `<link>`.
3. Dans le `body`, créez trois grandes zones :
    - un `<header>` contenant le logo (une image) et le nom du commerce ;
    - un `<nav>` contenant le menu sous forme de liste `<ul>` / `<li>` avec un lien `<a>` vers chaque page ;
    - un `<footer>` avec l'adresse, les horaires résumés, un lien « Mentions légales » (vers `#`) et « © 2026 Nom du commerce ».
4. Entre le menu et le pied de page, chaque page aura ses propres `<section>` (étape 2).
5. Donnez à chaque `<title>` un texte différent et précis, par exemple « Nos pains – Boulangerie Dupont ».

Vérification : ouvrez la page dans le navigateur et cliquez sur chaque lien du menu. Aucun lien ne doit être cassé.

Étape 2 — Contenus des pages

Remplissez chaque page avec de vrais textes (pas de « lorem ipsum ») et une hiérarchie de titres logique : un seul `<h1>` par page, puis des `<h2>` pour les sections, des `<h3>` pour les sous-parties.

Accueil (`index.html`)

1. Une `<section class="banniere">` avec le `<h1>` (nom du commerce), le slogan et un lien « Découvrir nos produits » vers `produits.html`.
2. Une section « Qui sommes-nous ? » : 2 paragraphes `<p>` et une image.
3. Une section « Nos points forts » : 3 blocs, chacun avec un `<h3>` et une phrase (ex. : fait maison, produits locaux, livraison).
4. Une section « Nos coups de cœur » : 3 cartes produits (même modèle que ci-dessous).

Produits (`produits.html`)

1. Classez vos produits en 2 ou 3 catégories, une `<section>` par catégorie avec un `id` (ex. : `id="pains"`).
2. En haut de page, une petite liste de liens vers ces ancres (`<a href="#pains">Nos pains</a>`).
3. Au moins 6 cartes au total, toutes construites **exactement** sur ce modèle :

```
<div class="carte">
    <img src="images/pain-campagne.jpg" alt="Pain de campagne au levain" />
    <h3>Pain de campagne</h3>
    <p class="description">Au levain naturel, cuit au feu de bois.</p>
    <p class="prix">4,20 €</p>
</div>
```

Contact (`contact.html`)

1. Les coordonnées : adresse, téléphone, e-mail.
2. Les horaires sous forme de liste `<ul>`, un jour par `<li>`.
3. Un lien « Voir sur le plan » vers Google Maps, qui s'ouvre dans un nouvel onglet (`target="_blank"`) et affiche une infobulle (`title`).
4. Le formulaire (étape 3).

Étape 3 — Formulaire de contact

Sur `contact.html`, créez un formulaire qui contient :

- nom et prénom (champs texte, obligatoires) ;
- e-mail (`type="email"`, obligatoire) ;
- téléphone (`type="tel"`, facultatif) ;
- objet de la demande : liste déroulante `<select>` avec au moins 3 choix adaptés à votre commerce (ex. : commande spéciale, renseignement, réclamation) ;
- message (`<textarea>`, obligatoire) ;
- une case à cocher « J'accepte que mes données soient utilisées pour me recontacter » ;

- un bouton « Envoyer ».

Chaque champ a un `<label>` relié par `for` / `id`, et surtout un attribut `name` clair, en minuscules et sans accent.

Pour l'instant, cliquer sur « Envoyer » affiche une erreur « fichier introuvable » : c'est normal, `traitement.php` n'existe pas encore. Vérifiez en revanche que le navigateur bloque l'envoi si un champ obligatoire est vide.

Étape 4 — Mise en forme CSS

Tout se passe dans `css/style.css`. Avancez du général au particulier et rafraîchissez le navigateur après chaque règle.

1. **Base de la page (`body`)** : police principale (`font-family`), taille du texte (`font-size`), couleur du texte et couleur de fond de votre identité.
2. **Titres** : une police ou une couleur distincte pour `h1` et `h2`, tailles en `em`.
3. **Largeur du contenu** : limitez les sections à une largeur fixe (ex. : `width: 1100px;`) et centrez-les avec `margin: auto;`.
4. **Bannière** : image de fond (`background-image`), texte centré (`text-align`), marges intérieures (`padding`) généreuses.
5. **Menu** : supprimez les puces de la liste et le soulignement des liens (`text-decoration: none;`) ; ajoutez un effet au survol avec `a:hover` (couleur ou soulignement).
6. **Cartes produits** : bordure (`border`), coins arrondis (`border-radius`), ombre (`box-shadow`), marges intérieures ; le prix en gras et dans votre couleur d'accent ; les images à `width: 100%;`.
7. **Formulaire** : champs sur toute la largeur, bordure fine, coins arrondis ; bouton aux couleurs du commerce avec un effet au survol.
8. **Pied de page** : fond foncé, texte clair, petite taille de police.

Règles de qualité :

- utilisez des classes (`.carte`, `.prix`, `.banniere`…) plutôt que de styliser toutes les balises `div` ;
- au plus 3 couleurs principales, toujours les mêmes codes hexadécimaux ;
- regroupez vos règles par zone avec des commentaires (`/* ===== MENU ===== */`).

Étape 5 — Mise en page avec Flexbox

Jusqu'ici, vos blocs s'empilent les uns sous les autres. Flexbox va les placer côte à côte. Rappel : `display: flex;` s'applique au **conteneur**, et ce sont ses enfants directs qui s'organisent. Si besoin, ajoutez dans le HTML un `<div class="...">` qui sert uniquement de conteneur.

Zone	Conteneur	Objectif	Propriétés à utiliser
En-tête	`header`	Logo et nom du commerce alignés sur une ligne	`display: flex`
Menu	`nav ul`	Liens sur une ligne, centrés, bien espacés	`justify-content: center` ou `space-around`
Points forts	`div.points-forts`	3 blocs côte à côte, de même largeur	`flex: 1` sur chaque bloc
Cartes produits	`div.grille`	3 cartes par ligne, retour à la ligne automatique	`flex-wrap: wrap`, `justify-content: space-between`, `width: 30%` sur `.carte`
Contact	`div.contact`	Coordonnées à gauche, formulaire à droite, plus large	`flex: 1` et `flex: 2`
Pied de page	`footer`	3 colonnes : adresse, horaires, liens	`display: flex`, `justify-content: space-between`

Vérification : réduisez la fenêtre du navigateur. Vous constatez que les cartes deviennent trop étroites et que le menu déborde : c'est ce que vous corrigerez à l'étape 6.

Étape 6 — Responsive avec les media queries

Toutes les règles de cette étape vont dans `css/responsive.css`, chargée **après** `style.css` : ses règles remplacent donc les précédentes quand la condition est remplie. Vérifiez aussi que la balise `<meta name="viewport">` de l'étape 1 est bien présente sur les 3 pages, sinon le téléphone affichera une version ordinateur miniature.

Traitez deux points de rupture :

Tablette : jusqu'à 1024px

- le contenu ne doit plus avoir de largeur fixe : remplacez la largeur de 1100px par `width: 90%;` ;
- les cartes passent à 2 par ligne (`width: 48%;`).

Smartphone : jusqu'à 600px

- en-tête et menu passent en colonne (`flex-direction: column;`), liens centrés ;
- les points forts, les cartes et les colonnes du pied de page s'empilent : 1 par ligne (`flex-direction: column;` ou `width: 100%;`) ;

- sur la page contact, le formulaire passe **au-dessus** des coordonnées grâce à `order` ;
- tailles de titres réduites, zone cliquable des liens et du bouton agrandie (`padding`).

Vérification : dans le navigateur, ouvrez les outils de développement (F12 ou Cmd+Option+I), activez le mode « appareil » et testez chaque page sur un iPhone, une tablette et un écran large. Aucun texte ne doit déborder et il ne doit jamais y avoir de défilement horizontal.

Rendu, évaluation et bonus

Rendez le dossier complet compressé en `.zip`, nommé `nom-prenom-commerce.zip`, à la date indiquée par l'enseignant. Ce dossier sera votre point de départ au module PHP : conservez-en une copie.

Avant de rendre, vérifiez :

- les 3 pages s'ouvrent et tous les liens du menu fonctionnent ;
- en-tête, menu et pied de page sont identiques sur les 3 pages ;
- toutes les images s'affichent et ont un `alt` ;
- aucun attribut `style` dans le HTML ;
- chaque champ du formulaire a un `name` ;
- le site est lisible sur smartphone, sans défilement horizontal ;
- le code est indenté et commenté.

Bonus :

- une 4e page « À propos » (histoire du commerce, équipe sous forme de cartes) ;
- une police Google Fonts intégrée localement (google webfonts helper) ;
- le lien du menu correspondant à la page en cours mis en valeur (classe `actif`) ;
- un badge « Nouveau » ou « Promo » sur certaines cartes, positionné en haut de l'image ;
- un code HTML sans erreur au validateur du W3C.